Cairo University
Faculty of Computers and Artificial Intelligence
Machine Learning Course

Lab 6 - Decision Tree

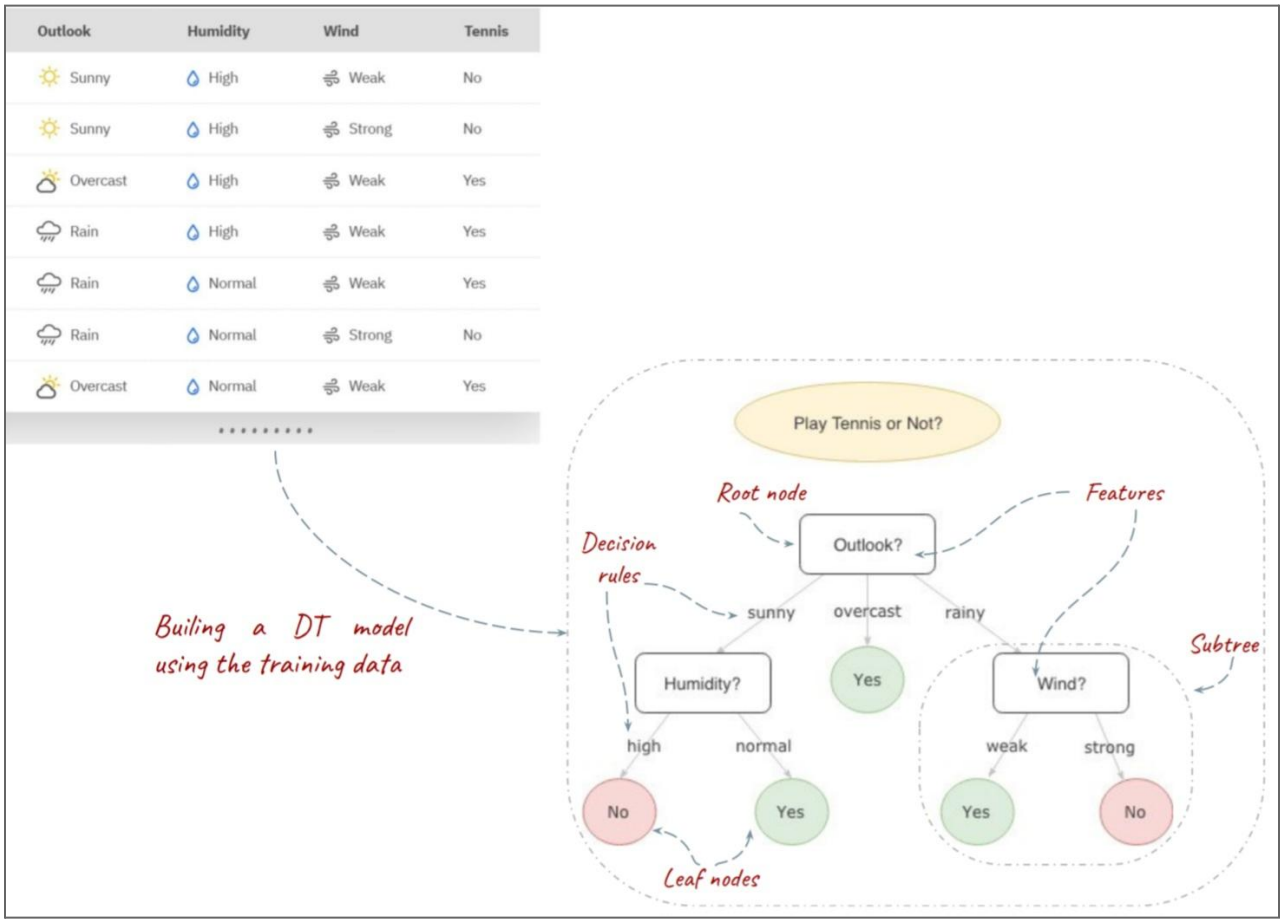
Lab Outline:

- Summary of decision trees
- Decision tree classification example in Python
- Practice

Summary of Decision Trees:

Decision Tree (DT) is a ***non-parametric supervised*** machine learning algorithm used for both ***classification and regression***. Its goal is to create a model that predicts the class/value of the target by learning simple decision rules inferred from the features.

The DT model is a ***flowchart-like tree structure*** where each internal node represents a feature and each branch represents a decision rule. The leaf nodes represent classes as they form homogenous subsets of the dataset.



In general, decision tree learning employs a divide and conquer strategy by conducting a greedy search to identify the optimal split points within a tree. The process of splitting is repeated in a top-down, recursive manner ***"recursive partitioning"*** until all records (or the majority) have been classified.

One of the popular decision tree algorithms is ***"Iterative Dichotomiser 3"*** (ID3). Its basic steps are:

1. **Evaluate how well each feature can split the current set of records using a certain criterion/metric.**
2. **Select the best feature to split this set and make a decision node for it** *(with branches corresponding to the feature's categories)***.**
3. **Split the current set into smaller subsets** *(each subset is a filtered version of the set's records based on the categories of the feature)***.**
4. **Repeat this process recursively for each subset using the remaining features** *(the subset's records become the current set)* **until all records in the subsets have the same class** *(or other stopping criteria)***.**

In order to evaluate how well each attribute (feature) can split the records and select the best one, we need to use a certain criterion. ID3 leverages two ***metrics to evaluate candidate splits.*** These metrics are:

- **Entropy:** measures the ***impurity*** of the set. Entropy values fall between 0 and 1. If all samples in the set *S* belong to one class, the entropy will be zero. On the other hand, if half of the samples are classified as one class and the other half are in another class, the entropy will be at its highest (one). ***Therefore, the attribute with the smallest amount of entropy should be selected.***

$$Entropy(S) = \sum_{i=1}^{c} -p_i \log_2 p_i$$

where ***S*** represents the set whose entropy is calculated, ***c*** represents the classes in the data, and $\boldsymbol{p_i}$ represents the proportion of records that belong to class ***i*** to the number of total records in ***S***.

- **Information gain:** represents the difference in entropy before and after a split on a given attribute, i.e. how much the impurity was reduced after splitting *S* on attribute *A*. ***Therefore, the attribute with the highest information gain will produce the best split.***

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

where ***S*** represents the set whose entropy is calculated, ***A*** represents a specific attribute, S_v represents the subset of records of ***S*** in which ***A***'s value is ***v***, and ***|S|*** represents the number of records in ***S***.

Decision Tree Classification Example in Python:

Recall the steps of building machine learning models from the previous labs:

1. **Analyze** the dataset
2. Perform data **preprocessing** *(including **splitting** into train and test sets)*
3. **Fit** the model to the training data
4. **Evaluate** the fitted model on the test data
5. Generate **predictions** for new data

After applying the first three steps on the "acceptance_data", we can use sklearn's DecisionTreeClassifier as follows:

```
# Step 3
from sklearn import tree

model = tree.DecisionTreeClassifier(criterion="entropy", max_depth=3)
model.fit(X_train, y_train)
```

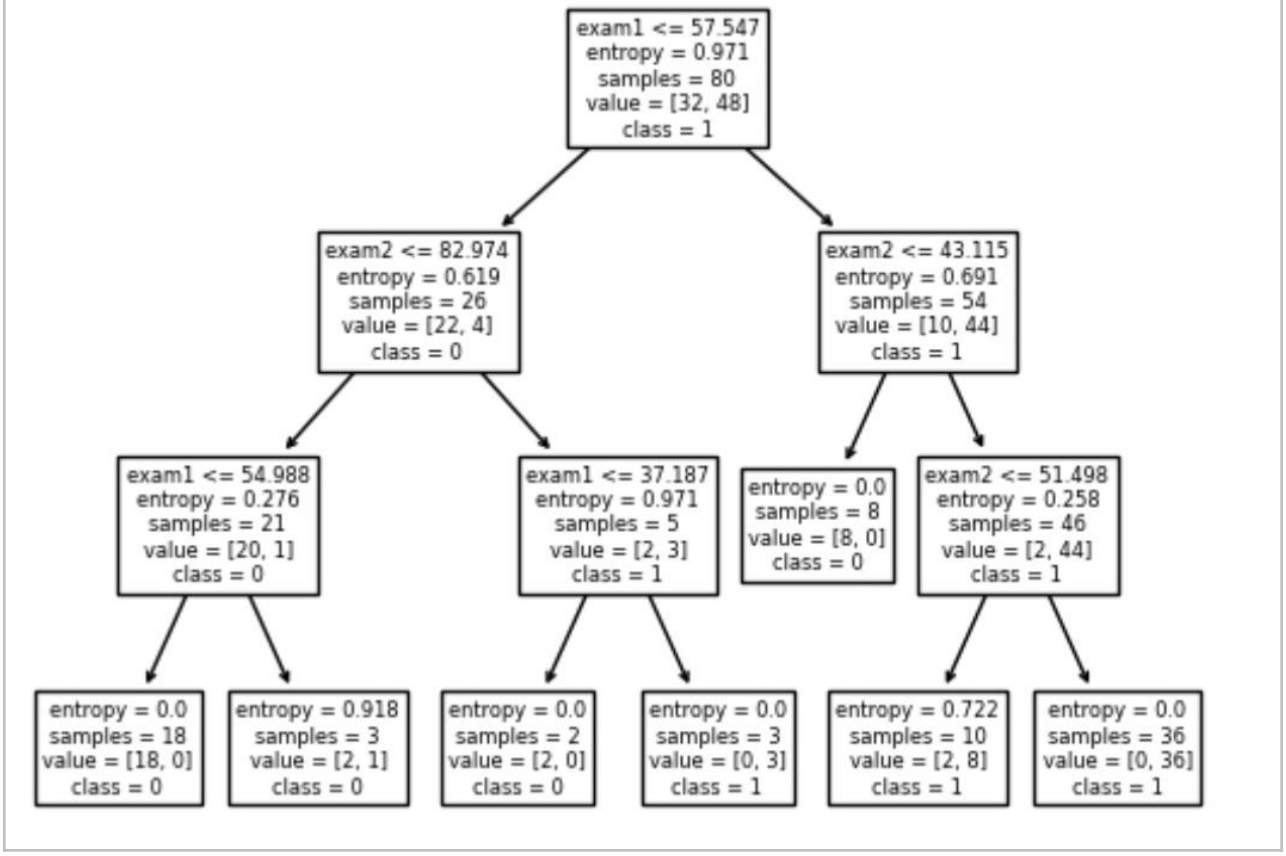
```
DecisionTreeClassifier(criterion='entropy', max_depth=3)
```

There are several hyperparameters that we can set in sklearn's DecisionTreeClassifier such as the split evaluation criterion (“gini”, “entropy”, “log_loss”), the maximum allowed depth of the tree, the minimum number of samples required to split an internal node, the minimum number of samples required to be at a leaf node, the number of features to consider when looking for the best split, the random state, and so on.

```
# Step 4
y_pred = model.predict(X_test)
print("Acc. of DT: ", model.score(X_test,y_test))
```

```
Acc. of DT:  0.8
```

```
tree.plot_tree(model, feature_names=X_train.columns,class_names=['0','1'])
plt.show()
# You can get a text representation of the rules using tree.export_text()
```



Since the features are numeric, the DT needs thresholds to create the splits. It can consider every possible value or, in this case, consider every midpoint of adjacent values after sorting.

In the tree in the previous image, notice how features are repeated as decision nodes with splits at different thresholds, and notice that not all leaves have pure sets.

```
# Model Info
print("Tree depth:", model.tree_.max_depth)
print("Total number of nodes:", model.tree_.node_count)
```

```
print("Number of leaf nodes:", model.tree_.n_leaves)
```

```
Tree depth: 3
Total number of nodes: 13
Number of leaf nodes: 7
```

Check out these tips before using DTs.

Practice:

Given a simple implementation of the decision tree algorithm from scratch in the file "Lab 6 (Task).ipynb", read the code and solve the tasks in the file.